SCALE FOR PROJECT FT_TRANSCENDENCE

You should evaluate 2 students in this team

Git repository

`git@vogsphere.42beiru`

Introduction

Please comply with the following rules:

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.

- Identify with the student or group whose work is evaluated the possible dysfunctions in their project. Take the time to discuss and debate the problems that may have been identified.

- You must consider that there may be differences in how your peers have understood the project's instructions and the scope of its functionalities.
Always keep an open mind and grade them as honestly as possible.
The pedagogy is only effective if the peer evaluation is done seriously.

Guidelines

- Only grade the work that is in the Git repository of the evaluated student or group.

- Verify each step carefully and test thoroughly.

- The project must reach at least 14 module points to pass.

- Check the README.md for the list of chosen modules and their justifications.

- All team members must be present and participate in the review.

- Double-check that the Git repository belongs to the student(s). Ensure that the project is the one expected. Also, check that 'git clone' is used in an empty folder.

- Check carefully that no malicious aliases was used to fool you and make you evaluate something that is not the content of the official repository.

- To avoid any surprises and if applicable, review together any scripts used to facilitate the grading (scripts for testing or automation).

- If you have not completed the assignment you are going to evaluate, you have to read the entire subject prior to starting the evaluation process.

- Use the available flags to report an empty repository, a non-functioning program, a Norm error, cheating, and so forth.
In these cases, the evaluation process ends and the final grade is 0, or -42 in case of cheating. However, except for cheating, students are strongly encouraged to review together the work that was turned in, in order to identify any mistakes that shouldn't be repeated in the future.

Attachments

subject.pdf

Preliminaries

Verify team presence, roles, and project understanding before proceeding with technical evaluation.

Team Presence

Verify that ALL team members (4-5 people) are present for the evaluation.

Are all team members present?

If not all members are present, the review stop here.

✓ Yes ✕ No

Individual Contributions

Ask EACH team member individually to explain:

- Their role in the project (PO, PM, Tech Lead, Developer)
- Their specific contributions and what they worked on
- At least one feature or module they personally implemented

Can each team member clearly explain their role and contributions?

If any team member cannot explain their role or contributions, this indicates a problem with team collaboration and individual participation.

✓ Yes ✕ No

README Verification

Verify the README.md contains ALL required sections:

- Project name and description
- Team members with assigned roles (PO, PM, Tech Lead, Developers)
- Project management approach (how work was organized)
- Technologies used with justifications
- Database schema
- List of features and who implemented them
- Chosen modules with justifications and point calculation
- Individual contributions of each member

Is the README.md complete with all required sections?

A missing or incomplete README will significantly impact the evaluation.

✓ Yes ✕ No

Project Coherence

Ask at least two different team members to explain:

- The project concept and what it does
- The main technologies used and why
- How the team coordinated the work

Can different team members explain the project coherently?

This verifies that all team members understand the project as a whole, not just their individual parts.

✓ Yes ✕ No

Git History

Verify team collaboration in Git:

- Repository shows commits from all team members
- Commit messages are clear and meaningful
- Work is distributed among team members

Ask a team member to show the commit history.
Does the Git history show proper team collaboration?

The Git history should reflect genuine teamwork, not just one person committing everyone's work.

✓ Yes ✕ No

General Requirements

Verify the mandatory technical requirements are met.

Architecture Components

Does the project have all three required components:

- Frontend
- Backend
- Database

Ask different team members to briefly explain each component.

All three components must be present and functional for the project to be considered complete.

✓ Yes ✕ No

Deployment

Can the entire application be deployed with a containerization solution (Docker, Podman, etc.) using a single command?

Ask a team member to demonstrate the deployment.

The deployment should be straightforward and work without manual intervention.
Verify that all services start correctly and the application is accessible.

✓ Yes ✕ No

Browser Compatibility

Does the application run on the latest stable Google Chrome without errors or warnings in the console?

Open Chrome DevTools and verify the console.

There should be no errors or warnings visible in the browser console.
Minor warnings from third-party libraries may be acceptable if explained.

✓ Yes ✕ No

Privacy Policy and Terms of Service

Are both Privacy Policy and Terms of Service pages accessible and contain relevant content?

These pages must:

- Be easily accessible from the application (e.g., footer links)
- Contain relevant and appropriate content for the project
- Not be placeholder or empty pages

Missing or inadequate Privacy Policy/Terms of Service pages will result in project rejection.

✓ Yes ✕ No

Technical Requirements

Verify the technical implementation meets the mandatory requirements.

Frontend Responsiveness

Is the frontend clear, responsive, and accessible across different devices?

Test on at least two different screen sizes (desktop and mobile/tablet).

The interface should adapt properly to different screen sizes and remain usable on both desktop and mobile devices.

✓ Yes ✕ No

Styling Solution

Is a CSS framework or styling solution used?
Examples: Tailwind CSS, Bootstrap, Material-UI, Styled Components, etc.

Ask a team member to show examples of its usage in the code.

The project must use a modern CSS framework or styling solution.
Plain CSS alone is not sufficient.

✓ Yes ✕ No

Environment Variables

Are credentials properly secured:

- .env file exists and is in .gitignore
- .env.example file is provided
- No sensitive credentials in the repository

Verify these security practices are followed.

Any credentials, API keys, or sensitive information found in the repository (outside of .env) will result in immediate project failure.

✓ Yes ✕ No

Database Design

Does the database have a clear schema with well-defined relations?

Ask a team member to show and explain the database schema (from README or directly).

The database should be properly structured with clear relationships between tables/collections.

✓ Yes ✕ No

Authentication Security

Does the user management system provide secure authentication with:

- Email and password signup/login
- Properly hashed and salted passwords

Ask the team to explain their password security implementation.

Passwords must never be stored in plain text. The team should be able to explain their hashing/salting approach.

✓ Yes ✕ No

Form Validation

Are all forms and user inputs validated in both frontend AND backend?

Test with invalid inputs (empty fields, wrong formats, SQL injection attempts, XSS attempts, etc.) to verify validation.

Both client-side and server-side validation must be present.
Server-side validation is critical for security.

✓ Yes ✕ No

Secure Connections

Is HTTPS used for all backend connections?

Verify the connection is secure (check browser address bar).

All communication between frontend and backend must be encrypted using HTTPS.

✓ Yes ✕ No

Modules Verification

Verify the chosen modules are properly implemented and reach the required 14 points minimum.

Module Points

Review the README.md modules section:

- List all claimed modules (Major = 2pts, Minor = 1pt)
- Calculate the total points

Does the project claim at least 14 points from modules?

The team must have selected modules totaling at least 14 points.
This is verified in the README before testing the actual implementation.

✓ Yes ✕ No

Major Modules

For EACH Major module (2 points) claimed:

- Ask the team to demonstrate it
- Verify it's fully implemented and functional
- Check it meets Major module complexity requirements
- Confirm it adds real value to the project
- Verify module dependencies are met (e.g., gaming modules require a game)

Are all claimed Major modules properly implemented and functional?

IMPORTANT: Only count working modules. Non-functional or incomplete modules = 0 points.

You must verify EACH major module individually. A module that doesn't work or doesn't meet the requirements specified in the subject PDF gets 0 points.

✓ Yes ✕ No

Minor Modules

For EACH Minor module (1 point) claimed:

- Ask the team to demonstrate it
- Verify it's fully implemented and functional
- Check it adds meaningful value to the project
- Verify module dependencies are met

Are all claimed Minor modules properly implemented and functional?

IMPORTANT: Only count working modules. Non-functional or incomplete modules = 0 points.

You must verify EACH minor module individually. A module that doesn't work or doesn't meet the requirements specified in the subject PDF gets 0 points.

✓ Yes ✕ No

Modules of Choice

If custom "Modules of choice" are claimed:

- Verify proper justification in README explaining:
  - Why they chose this module
  - What technical challenges it addresses
  - How it adds value to the project
  - Why it deserves Major/Minor status
- Confirm it's not a shortcut (must demonstrate technical complexity)
- Check relevance to project context
- Ensure technical skill is demonstrated

Are custom modules properly justified and implemented?

Custom modules must be substantial and demonstrate real technical skill.
Trivial features or shortcuts will not be accepted.

✓ Yes ✕ No

Code Quality

Assess the overall code quality, organization, and team understanding.

Code Structure

Is the code reasonably well-organized and readable?

- Clear file/folder structure
- Code is understandable
- No major code quality issues
- Consistent coding style

Review some code files to assess organization.

The code doesn't need to be perfect, but it should be organized and maintainable. Major code quality issues may indicate problems.

✓ Yes ✕ No

Technical Decisions

Can the team explain their technical choices?
Ask about:

- Why they chose their tech stack
- How they structured the application
- Challenges faced and solutions implemented
- Trade-offs made during development

Does the team demonstrate understanding of their project?

The team should be able to justify their technical decisions and demonstrate understanding of the technologies they used.

✓ Yes ✕ No

Teamwork Evidence

Is there evidence of effective teamwork?

- All members contributed (check Git history)
- Team members can explain each other's work
- Work appears coordinated and integrated
- README shows clear work distribution
- No single person did all the work

Does the project demonstrate effective team collaboration?

This is a group project. All members must have contributed meaningfully.

✓ Yes ✕ No

Functionality

Test the overall functionality and stability of the application.

Stability and Functionality

Is the application functional and reasonably stable?

- No critical bugs or crashes during demonstration
- Main features work as expected
- Basic error handling is present
- User experience is acceptable
- Multi-user support works (multiple users can use the app simultaneously)

Test the main features of the application.

The application should be stable enough for a demonstration.
Minor bugs are acceptable, but critical failures are not.

✓ Yes ✕ No

Overall Quality

Does the project demonstrate effort and learning?

- Shows understanding of web development concepts
- Goes beyond minimal requirements
- Team learned new technologies or concepts
- Project shows creativity or interesting implementation
- The project concept is well-executed

Does the project reflect genuine effort and learning?

This project is about learning and demonstrating skills.
The project should show that the team invested effort and learned.

✓ Yes ✕ No

Final Verification

Final checks to ensure the project meets all requirements.

Final Module Count

Final module score calculation:

- Count ONLY the modules that were successfully demonstrated and work properly
- Major modules = 2 points each
- Minor modules = 1 point each
- Non-functional or incomplete modules = 0 points

Does the total of VALIDATED modules reach at least 14 points?

CRITICAL: Be fair but strict. Only count modules that actually work and meet the requirements specified in the subject PDF.

If the project does not reach 14 points from validated modules, it fails.

✓ Yes ✕ No

Project Success

Overall assessment:

- Is the mandatory part complete and functional?
- Did all team members contribute meaningfully?
- Can the team explain their work and decisions?
- Does the project meet the subject requirements?
- Is the README complete and accurate?

Would you consider this a successful group project?

This is your final assessment of whether the project meets the standards expected for this assignment.

✓ Yes ✕ No

Bonus

Evaluate the bonus part if, and only if, the mandatory part has been entirely and perfectly done, and the error management handles unexpected or bad usage. In case all the mandatory points were not passed during the defense, bonus points must be totally ignored. Bonus points are awarded for modules beyond the required 14 points. Each additional validated module adds bonus points: - Major module = 2 bonus points - Minor module = 1 bonus point

Extra Modules

You must now verify any extra modules beyond the required 14 points.

For each extra module:

- Verify it's fully functional
- Confirm it meets the module requirements from the subject PDF
- Check that it adds value to the project
- Ensure proper justification in README

Count the number of extra validated modules:

- Major modules = 2 bonus points each
- Minor modules = 1 bonus point each

A module is considered valid under the following criteria:

- There are no issues with the proper functioning of the module
- It meets all requirements specified in the subject PDF
- No errors are visible
- A comprehensive explanation allows for detailed understanding

Bonus points are awarded based on the number of validated extra modules,
with a maximum of 5 bonus points
(e.g. 5 minor modules, or 2 major modules + 1 minor module).

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don't forget to check the flag corresponding to the defense

✓ Ok | ★ Outstanding project

Empty work | Incomplete work | Cheat | Crash | Incomplete group

Concerning situation | Forbidden function | Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation